Objective(s):

a. Implement an undirected, unweighted graph with three internal representations.
b. Use depth first search to answer reachability queries and to traverse a graph.
c. Detect a cycle in an undirected graph stored as an adjacency map.
d. Perform depth first search directly on an adjacency matrix.

General rules:

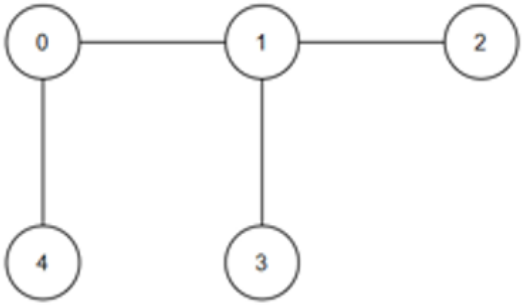
e. Vertices initially use labels 0 through n - 1. Task 1 and Task 2 graphs are undirected and unweighted.
f. Keep neighbors in their insertion order. DFS must visit neighbors in that order.
g. Do not store duplicate edges. An invalid or missing vertex gives false for hasEdge and hasPath, and an empty list for neighborsOf.
h. • printGraph prints one vertex per line in the form value -> neighbors. The exact spacing may differ.

Task 1: Graph representations

use GraphNode as the reference implementation; implement GraphList and GraphMap.

1.1 GraphNode

GraphNode stores a list of Node objects. Each Node has an integer value and a list of its neighboring Node objects. This is the reference representation for the lab. Build this class first, then implement the equivalent behavior in GraphList and GraphMap.




```
List<int[]> edges = Arrays.asList(
    new int[]{0, 1}, new int[]{0, 4},
    new int[]{1, 2}, new int[]{1, 3}
);
GraphNode graph = new GraphNode(5, edges);
System.out.println(graph.hasEdge(1, 3)); // true
System.out.println(graph.hasPath(4, 2)); // true
graph.dfs(0);                            // 0 1 2 3 4
graph.addNode(5);                        // isolated vertex
```

Method	Required behavior
GraphNode(int n, List<int[]> edges)	Create vertices 0 through n - 1, then add every undirected edge from edges.
addNode(int value)	Add an isolated vertex when the value does not exist. Return the Node for that value.
addEdge(int a, int b)	Add one undirected edge. Add both endpoint vertices if needed.
neighborsOf(int value)	Return the neighboring Node objects. Node.toString() should show its value.
hasEdge / hasPath / dfs	Use node references internally. dfs takes a starting vertex value and prints each visited value once.
printGraph()	Print each Node followed by its neighboring Node values.

1.2 GraphList

GraphList uses an array of lists. The constructor receives n and the same list of edges used by the other two classes.

```
List<int[]> edges = Arrays.asList(
    new int[]{0, 1}, new int[]{0, 4},
    new int[]{1, 2}, new int[]{1, 3}
);
GraphList graph = new GraphList(5, edges);
System.out.println(graph.neighborsOf(1));
                                        // [0, 2, 3]
graph.dfs(0);
                                        // 0 1 2 3 4
```

Method	Required behavior
GraphList(int n, List<int[]> edges)	Create the adjacency-list array and add every edge from edges.
addEdge(int a, int b)	Add b to a's list and a to b's list.
neighborsOf / hasEdge / hasPath / dfs	Provide the same behavior as GraphNode with integer vertex labels.
printGraph()	Print every vertex from 0 through vertices - 1.

1.3 GraphMap

GraphMap uses a map from each vertex label to its adjacency list. Its constructor receives n and the same list of edges used by GraphNode and GraphList.

```
List<int[]> edges = Arrays.asList(
    new int[]{0, 1}, new int[]{0, 4},
    new int[]{1, 2}, new int[]{1, 3}
);
GraphMap graph = new GraphMap(5, edges);
graph.addVertex(5);     // isolated vertex
```

Method	Required behavior
GraphMap(int n, List<int[]> edges)	Create vertices 0 through n - 1, then add every undirected edge from edges.
addVertex(int value)	Add an isolated vertex only when it does not already exist.
addEdge(int a, int b)	Add an undirected edge and avoid duplicates.
neighborsOf / hasEdge / hasPath / dfs / dfsAll	Provide the same query and DFS behavior as the other classes. dfsAll starts a new line for each component.
printGraph()	Print each map key followed by its adjacency list.

Task 2: Cycle detection in GraphMap

Add boolean hasCycle() to GraphMap. Return true when the graph contains a cycle. Use DFS and remember the parent vertex. When visiting a neighbor that has already been visited and is not the parent, a cycle exists.

```
0 -- 1 -- 2 -- 3
|               |
+---------------+
```

The parent is the vertex from which DFS reached the current vertex. In an undirected graph, the edge back to the parent is expected and is not a cycle. A cycle exists only when the current vertex reaches a previously visited neighbor that is not its parent. For example, after DFS travels 0 -> 1 -> 2 -> 3, an edge from 3 to 0 closes a cycle.

Task 3: DFS on an adjacency matrix

In Lab12a_Graph1_XXYYYY.java, complete dfsMatrix(int[][] matrix, int start). The matrix represents a directed weighted graph. A value greater than 0 and less than INF represents an edge. dfsMatrix(matrix, start) must visit every vertex reachable from start at most once and print them in DFS visitation order.

Submit GraphNode_XXYYYY.java, GraphList_XXYYYY.java, GraphMap_XXYYYY.java, and Lab12a_Graph1_XXYYYY.java where. XX are the first two digit of your student id and YYYY are the last four.

Due date: TBA